Cenni di Algoritmi e Strutture Dati

Lezione 3 - Stack e Queue

Corso PON "Coding & App" - Istituto Superiore di Istruzione "Archimede", Treviglio

Cos’è una stack?

È una struttura dati che supporta almeno due principali operazioni:

- Push: inserisce un elemento nella struttura dati
- Pop: rimuove l’elemento più recentemente inserito

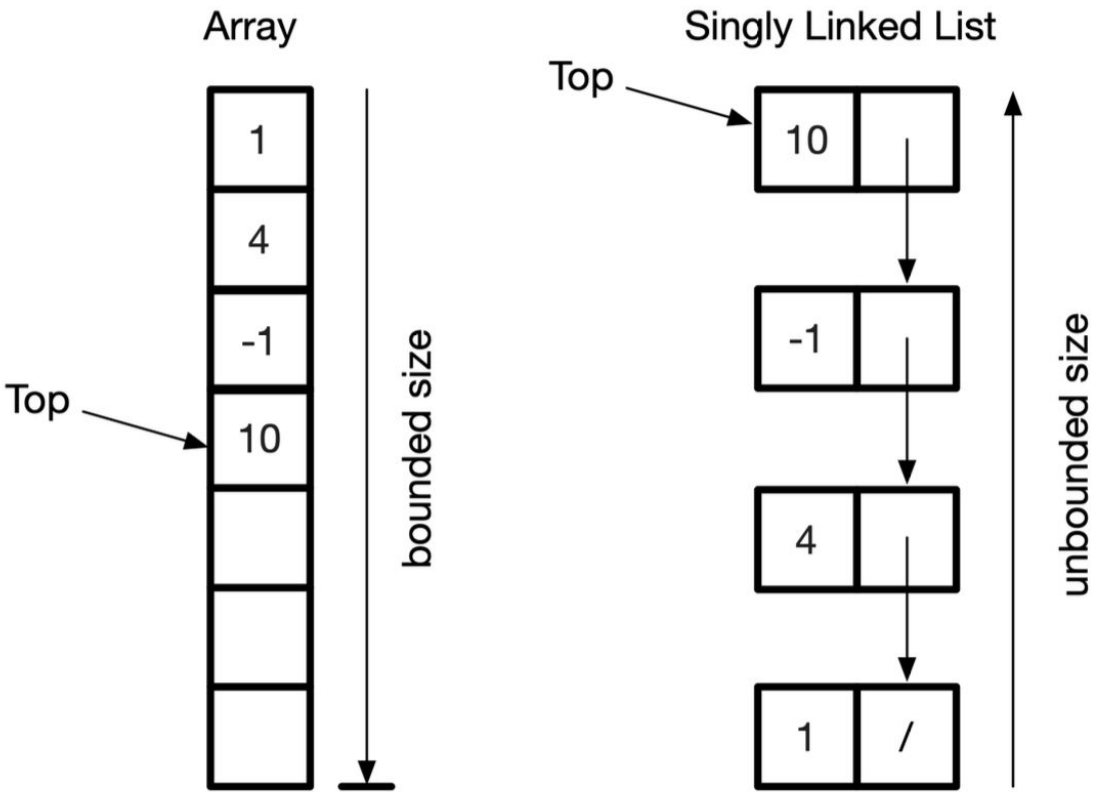
Intuitivamente, è una pila di elementi uno sopra l’altro, come a simulare una “pila di piatti” di un ristorante.

È una struttura con policy di esecuzione LIFO (Last In First Out). È utilizzata in diversi campi:

- Richiami alle funzioni;
- Editor di testo (copia, incolla, indietro...);
- Il parsing di sintassi (esempio apertura e chiusura delle parentesi in un programma)

Implementazione della struttura dati Stack

Array	SLList
POP: rimuove l'ultimo elemento nell'array	POP: rimuove la testa della lista
PUSH: inserisce l'elemento nella prima posizione libera	PUSH: inserisce l'elemento nella testa della lista
Pro: nessun uso sopraelevato di memoria (si memorizza solamente il valore)	Pro: nessun limite alla dimensione della stack
Contro: dimensione limitata per la stack	Contro: uso di Doubly o Circular?


Array
1
4
-1
Top
10
bounded size
Singly Linked List
Top
10
-1
4
1
/
unbounded size

PUSH e POP con array statici

```
function PUSH(STACK S, INT x)
    if S.top == S.length then
        error "overflow"
    else
        S.top = S.top + 1
        S.stack[S.top] = x
```

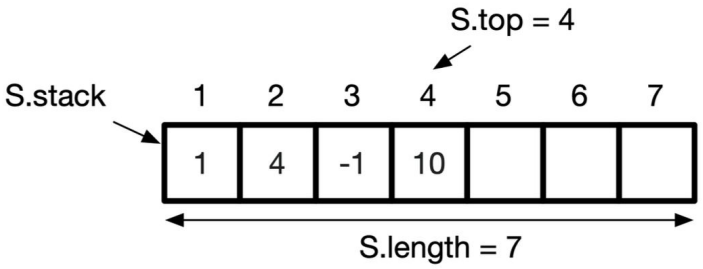
```
function POP(STACK S) → INT
    if S.top == 0 then
        error "underflow"
    else
        e = S.stack[S.top]
        S.top = S.top − 1
        return e
```

```java
public void push(int item) {
  if (count == items.length)
    throw new StackOverflowError();

  items[count++] = item;
}

2 usages
public int pop() {
  if (count == 0)
    throw new IllegalStateException();

  return items[--count];
}
```



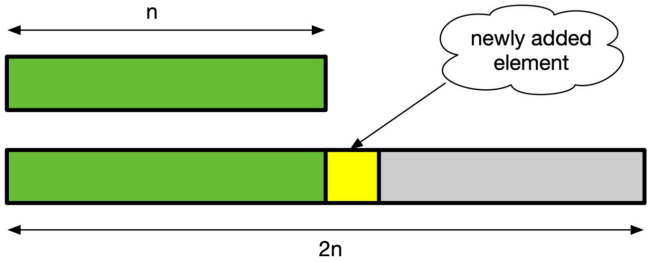
- Costo entrambe operazioni: **O(1)**
- Stack Overflow per chiamate errate di operazioni di push
- Stack Underflow per chiamate errate di operazioni di pop

Implementazione con Array Dinamico

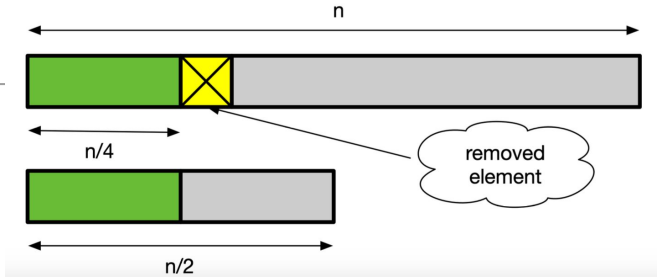
Implementazione di un array dinamico

strategia di resizing

Raddoppiare la dimensione dell'array quando non c'è più spazio



Dimezzare la dimensione dell'array quando sono occupati solamente ¼ delle posizioni disponibili



POP con Array Dinamico

```
1:  function POP(STACK S) → INT
2:      if S.top == 0 then
3:          error "underflow"
4:      else
5:          e = S.stack[S.top]
6:          S.top = S.top − 1
7:          if S.top ≤ ⌊S.length/4⌋ then
8:              n = S.length
9:              LET T[1 ··· ⌈n/2⌉] BE A NEW ARRAY
10:             for i = 1, ··· , ⌊n/4⌋ do
11:                 T[i] = S.stack[i]
12:             S.stack = T
13:             S.length = ⌈n/2⌉
14:         return e
```

ANALISI DEI COSTI

<u>Worst Case Scenario</u>: **O(n)**

Copia dell'array di dimensione n alla riga 10

<u>Best Case Scenario</u>: **O(1)**

Più di ¼ dello spazio utilizzato

PUSH con Array Dinamico

```
1:  function PUSH(STACK S, INT x)
2:      if S.top == S.length then
3:          n = S.length
4:          LET T[1 ··· 2n] BE A NEW ARRAY
5:          for i = 1, ··· , n do
6:              T[i] = S.stack[i]
7:          S.stack = T
8:          S.length = 2n
9:      S.top = S.top + 1
10:     S.stack[S.top] = x
```

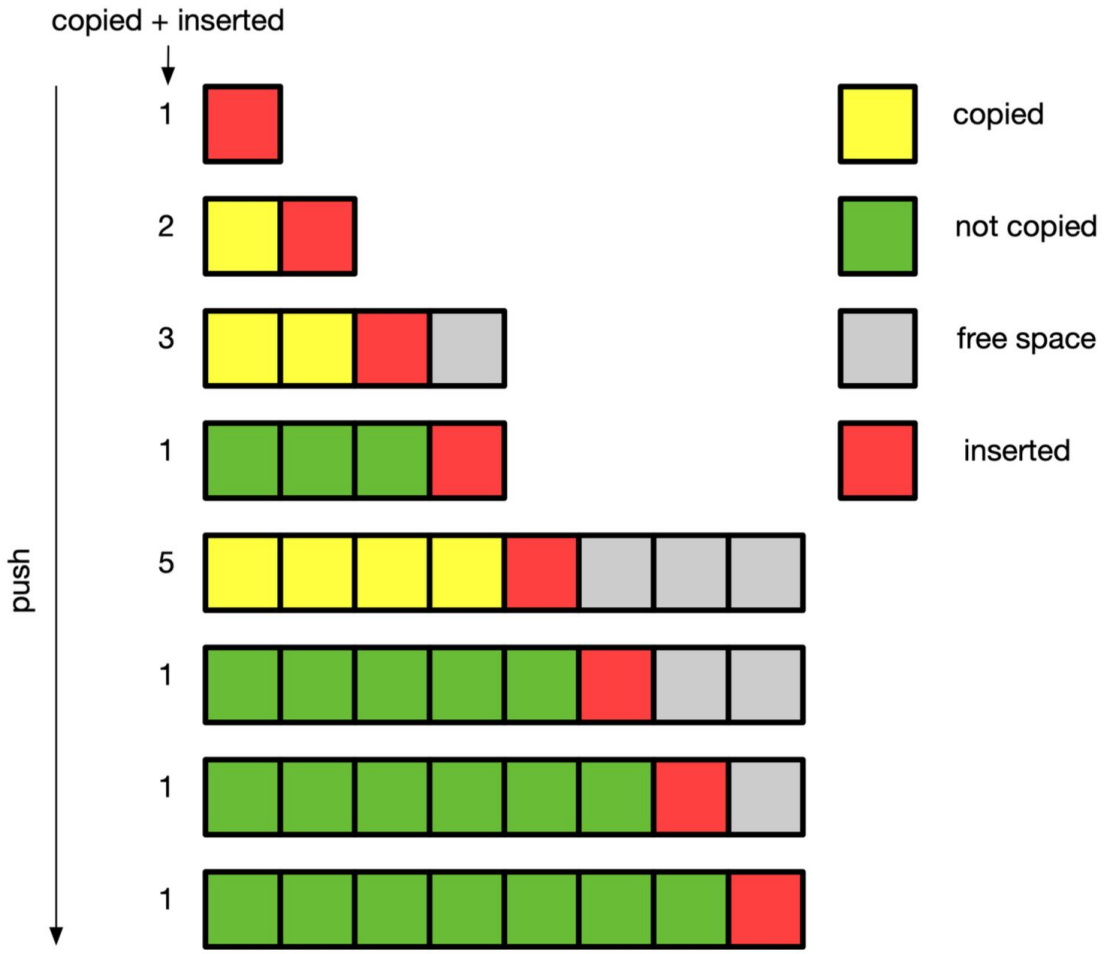
ANALISI DEI COSTI

<u>Worst Case Scenario</u>: **O(n)**

Copia dell'array di dimensione n alla riga 5

<u>Best Case Scenario</u>: **O(1)**

Array non pieno


copied + inserted
push
1
2
3
1
5
1
1
1
copied
not copied
free space
inserted

- What is the cost for n PUSH starting from an empty Stack?
  - Worst-case cost is $O(n^2)$ if we use upper bound $O(n)$
  - $O(n^2)$ is not a tight bound: we do not expand the array often
  - The cost of the i-th PUSH is in fact

$$c_i = \begin{cases} i & \text{if } i-1 \text{ is an exact power of 2} \\ 1 & \text{otherwise} \end{cases}$$

  - The total cost of n PUSH is then (aggregate method)

$$\sum_{i=1}^{n} c_i \leq n + \sum_{j=0}^{\log_2 n} 2^j = n + \frac{2^{\log_2 n+1} - 1}{2-1} = 3n - 1 = O(n)$$

Cos’è una queue?

È una struttura dati che supporta almeno due principali operazioni:

- Enqueue: inserisce un elemento alla fine della struttura dati
- Dequeue: rimuove l’elemento all’inizio della struttura dati

Intuitivamente, è una fila di elementi uno dopo l’altro, come a simulare una “fila di persone in attesa”.

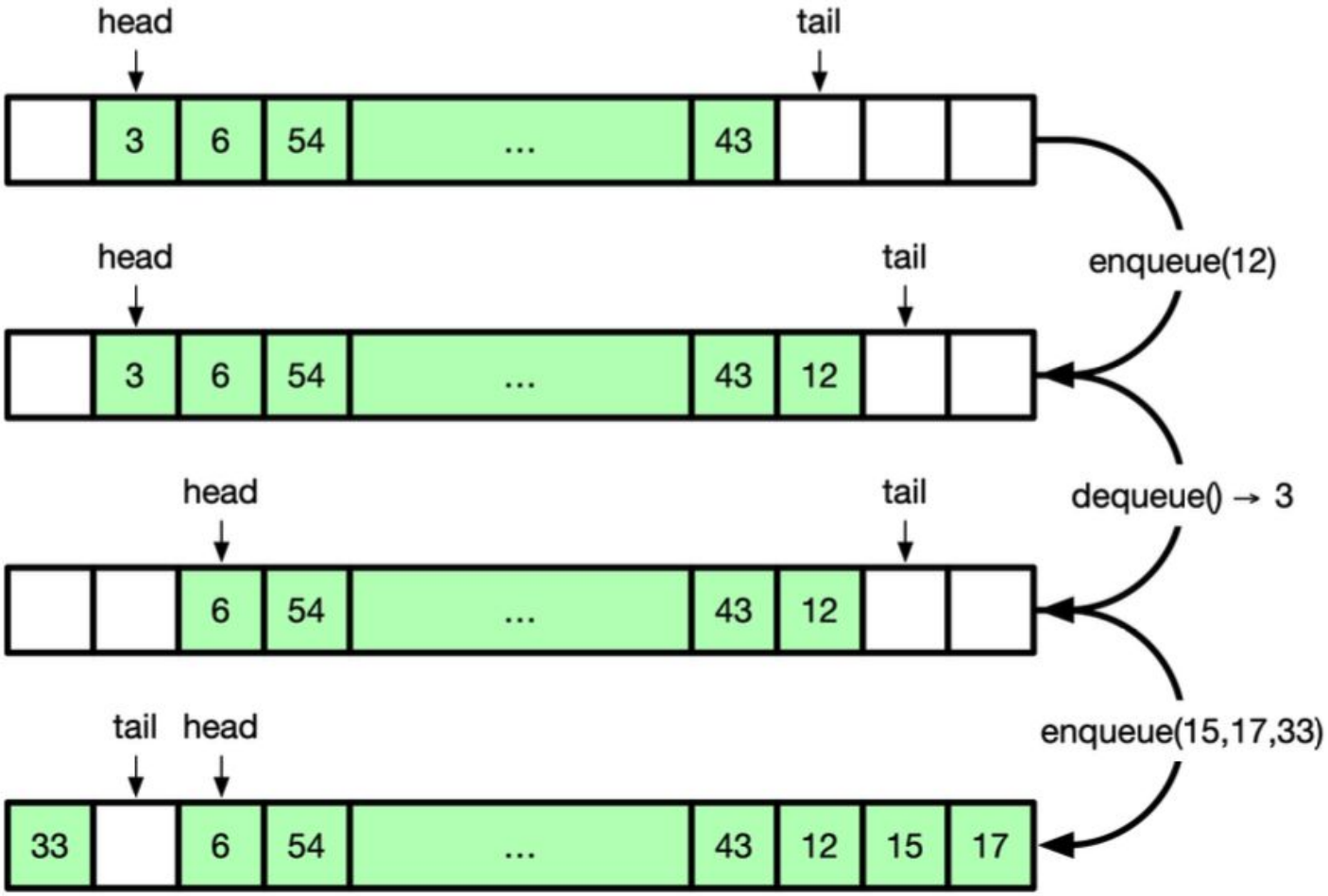
È una struttura con policy di esecuzione FIFO (First In First Out). È utilizzata in diversi campi:

- Breadth-first-search sui grafi;
- Programmazione di processi nei sistemi operativi.

Implementazione della struttura dati Queue

<table>
<tr><th>Circular Linked List</th><th>Singly Linked List</th><th>Array</th></tr>
<tr><td>Enqueue: inserimento nella coda della lista</td><td>Enqueue: inserimento nella coda della lista, aggiungendo un puntatore dalla coda</td><td>Enqueue: inserimento nella coda della lista</td></tr>
<tr><td>Dequeue: rimuove la testa della lista</td><td>Dequeue: rimuove la testa della lista</td><td>Dequeue: rimuove la testa della lista</td></tr>
<tr><td>Pro: struttura dinamica senza limiti</td><td rowspan="2">In TUTTI i casi, il costo è

O(1)</td><td>Pro: nessun utilizzo elevato di memoria (si memorizza solamente il valore)</td></tr>
<tr><td>Contro: utilizzo elevato di memoria (necessaria introduzione di 2 puntatori, oltre al dato)</td><td>Contro: memoria limitata</td></tr>
</table>


head
tail
3
6
54
...
43
enqueue(12)
head
tail
3
6
54
...
43
12
dequeue() → 3
head
tail
6
54
...
43
12
enqueue(15,17,33)
tail
head
33
6
54
...
43
12
15
17

```
function ENQUEUE(QUEUE Q, INT x)
    if Q.size == Q.length then
        error "overflow"
    Q, buf[Q.tail] = x
    Q.tail = (Q.tail % Q.length) + 1
    Q.size = Q.size + 1
```

```
function DEQUEUE(QUEUE Q) → INT
    if Q.size == 0 then          ▷ Empty Q
        error "underflow"
    x = Q.buf[Q.head]
    Q.head = (Q.head % Q.length) + 1
    Q.size = Q.size − 1
    return x
```

Costo di entrambe le funzioni: **O(1)**

Q.size è il numero di elementi in Q

tail (coda) punta alla prima posizione libera

L'operazione matematica **modulo** ci permette di scansionare circolarmente l'array

Come si potrebbe implementare la struttura dati Queue utilizzando un array circolare <u>dinamico</u>?

Esercizi

Esercizio 1

Scrivere un algoritmo che, dato in input una Singly Linked List di interi e un intero x, restituisca il numero di nodi contenenti il valore x.

Ipotizzare, poi, il costo computazionale dell’algoritmo.

Esercizio 1 - soluzione

```
function COUNT(LIST L, INT x)                ▷ Iterative solution
    n = 0
    while ( doL ≠ NIL)
        if L.key == x then
            n = n + 1
        L = L.next
```

```
function COUNT(LIST L, INT x)                ▷ Recursive solution
    if L == NIL then
        return 0
    else if L.key == x then
        return 1 + COUNT(L.next)
    else
        return COUNT(L.next)
```

Esercizio 2

Scrivere un algoritmo che, dato in input una Singly Linked List **L,** inverta la lista **L** e restituisca il puntatore alla nuova testa della lista.

Esempio:

L = 1 → 2 → 3 → 4

Lreverse = 4 → 3 → 2 → 1

Ipotizzare, poi, il costo computazionale dell'algoritmo.

Esercizio 2 - soluzione

```
function REVERSE(LIST L)
    prev = next = NIL
    while L ≠ NIL do
        next = L.next
        L.next = prev
        prev = L
        L = next
    return prev
```

Esercizio 3

Si consideri la seguente sequenza di caratteri
E A S * Y * Q U E * * * S T * * * I O * N * * *

A partire da una stack S **vuota**, un carattere **c** significa un'operazione di push(S,c) e un asterisco significa un'operazione di pop(S). Mostrare a video la sequenza di caratteri restituita dalle operazioni di **pop.**

A partire da una queue Q **vuota**, un carattere **c** significa un'operazione di enqueue(Q,c) e un asterisco significa un'operazione di dequeue(Q). Mostrare a video la sequenza di caratteri restituita dalle operazioni di **dequeue.**

Esercizio 4

Si consideri una Stack S che supporta le seguenti operazioni con costo computazionale O(1)

- PEEK(s), che restituisce il valore in cima ad S (senza rimuoverlo)
- ISEMPTY(S) vero se S è **vuota**, falso altrimenti
- PUSH (S, x), che inserisce x in cima alla Stack S
- POP (S) che rimuove e restituisce il valore in cima di S, sollevando un'eccezione se S1 è vuota

Scrivere un algoritmo che tenga traccia del valore minimo presente nella stack S1, utilizzando una seconda stack S2 che contenga, di volta in volta, come elemento in cima il minimo elemento di S1

Esercizio 4 - soluzione

```
function PUSH2(STACK S1, STACK S2, INT x)
    PUSH(S1,x)
    if ISEMPTY(S2) or TOP(S2) ≥ x then
        PUSH(S2,x)
```

```
function POP2(STACK S1, STACK S2) → INT
    if not ISEMPTY(S1) and TOP(S1) == TOP(S2) then
        POP(S2)
    return POP(S1)
```